Interaction Terms in Regression

General Principles

If you have a case where you believe the effect of one independent variable depends on the value of another independent variable, you can use regression analysis with interaction terms. In this approach, we extend the simple linear regression model to include an interaction term (a multiplication) between the two independent variables (see note on how this multiplication arises).

Considerations

Note

- We have the same assumptions as for Linear Regression for continuous variable.
- We wish to model the relationship between a dependent variable, Y, and an independent variable, X_1, whose effect varies as a function of a second independent variable X_2. To do this, we explicitly model the hypothesis that the slope between Y and X_1 depends on (i.e., is conditional on) X_2.
- For continuous interactions with normalized data, the intercept becomes the grand mean of the outcome variable.
- The interpretation of slopes estimates is more complex. The coefficient for a non-interaction term reflects the expected change in Y when X_1 increases by one unit, holding X_2 constant at its average value. The coefficient for the interaction term represents how the effect of X_1 on Y changes depending on the value of X_2, and vice versa, showing how the relationship between the two variables influences the outcome Y.
- Triptych plots are very handy for understanding the impact of interactions, especially when more than two interactions are present.

Example

Below is example code demonstrating Bayesian regression with an interaction term between two continuous variables using the BayesForge (BF) package. The data consist of three continuous variables (temperature, humidity, energy consumption), and the goal is to estimate the effect of the interaction between temperature and humidity on energy consumption. This example is based on McElreath (2018).

Python

```python
from BayesForge import bf

# Setup device-------------------------------------------------
m = bf(platform='cpu')

# Import Data & Data Manipulation ------------------------------------------------
data_path = m.load.tulips(only_path = True)
m.data(data_path, sep=';')
m.df['blooms_scaled'] = m.df.blooms / m.df.blooms.max()
m.df['water_scaled'] = m.df.water - m.df.water.mean()
m.df['shade_scaled'] = m.df.shade - m.df.shade.mean()
m.data_to_model(['blooms_scaled', 'water_scaled', 'shade_scaled'])


# Define model -------------------------------------------------
def model(blooms_scaled, shade_scaled, water_scaled):
    sigma = m.dist.exponential(1, name = 'sigma')
    bws = m.dist.normal(0, 0.25, name = 'bws')
    bs = m.dist.normal(0, 0.25, name = 'bs')
    bw = m.dist.normal(0, 0.25, name = 'bw')
    a = m.dist.normal(0.5, 0.25, name = 'a')
    mu = a + bw*water_scaled + bs*shade_scaled + bws*water_scaled*shade_scaled
    m.dist.normal(mu, sigma, obs=blooms_scaled)

# Run mcmc -------------------------------------------------
m.fit(model, progress_bar=False) # Optimize model parameters through MCMC sampling

# Summary -------------------------------------------------
m.summary()
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal
```

```
bf v 0.0.60 package loaded
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
a	0.36	0.03	0.32	0.40	5471.75	3276.59	1.0	0.0	0.0
bs	-0.11	0.03	-0.17	-0.06	4650.00	2891.92	1.0	0.0	0.0
bw	0.21	0.03	0.15	0.26	4258.47	2706.25	1.0	0.0	0.0
bws	-0.14	0.04	-0.21	-0.08	5112.89	3048.08	1.0	0.0	0.0
sigma	0.14	0.02	0.11	0.18	3624.46	3052.33	1.0	0.0	0.0

R

```r
library(BayesForge)
m <- importBF(platform = "cpu")

# Load csv file
m$data(m$load$tulips(only_path = TRUE), sep = ';')
m$df$blooms_scaled <- m$df$blooms / max(m$df$blooms)
m$df$water_scaled <- m$df$water - mean(m$df$water)
m$df$shade_scaled <- m$df$shade - mean(m$df$shade)
m$data_to_model(list('blooms_scaled', 'water_scaled', 'shade_scaled'))

# Define model ------------------------------------------------
model <- function(blooms_scaled, water_scaled, shade_scaled){
  # Parameter prior distributions
  sigma = m$dist$exponential(1, name = 'sigma')
  bws = m$dist$normal(0, 0.25, name = 'bws')
  bs = m$dist$normal(0, 0.25, name = 'bs')
  bw = m$dist$normal(0, 0.25, name = 'bw')
  a = m$dist$normal(0.5, 0.25, name = 'a')
  mu = a + bw*water_scaled + bs*shade_scaled + bws*water_scaled*shade_scaled
  # Likelihood
  m$dist$normal(mu, sigma, obs = blooms_scaled, name = 'blooms_scaled')
}

# Run mcmc ----------------------------------------------------
m$fit(model, progress_bar = FALSE) # Optimize model parameters through MCMC sampling

# Summary -----------------------------------------------------
m$summary() # Get posterior distributions
```

Julia

```
using PyBayesForge

# Setup device--------------------------------------------------
m = importBF(platform="cpu")

# Import Data & Data Manipulation ------------------------------------------------
data_path = m.load.tulips(only_path = true)
m.data(data_path, sep=';')
df = m.df
df["blooms_scaled"] = df["blooms"] / df["blooms"].max()
df["water_scaled"] = df["water"] - df["water"].mean()
df["shade_scaled"] = df["shade"] - df["shade"].mean()
m.data_to_model(["blooms_scaled", "water_scaled", "shade_scaled"])

# Define model -------------------------------------------------
@BF function model(blooms_scaled, shade_scaled, water_scaled)
    sigma = m.dist.exponential(1, name = "sigma")
    bws = m.dist.normal(0, 0.25, name = "bws")
    bs = m.dist.normal(0, 0.25, name = "bs")
    bw = m.dist.normal(0, 0.25, name = "bw")
    a = m.dist.normal(0.5, 0.25, name = "a")
    mu = a + bw*water_scaled + bs*shade_scaled + bws*water_scaled*shade_scaled
    m.dist.normal(mu, sigma, obs=blooms_scaled)
end

# Run mcmc -----------------------------------------------------
m.fit(model)  # Optimize model parameters through MCMC sampling

# Summary ------------------------------------------------------
m.summary() # Get posterior distributions
```

Mathematical Details

Frequentist formulation

We model the relationship between the input features (X_1 and X_2) and the target variable (Y) using the following equation:

$$Y_i = \alpha + \beta_1 X_{[1,i]} + \beta_2 X_{[2,i]} + \beta_3 X_{[1,i]} X_{[2,i]} + \epsilon_i$$

Where:

- Y_i is the dependent variable for observation i.
- α is the intercept term.
- $X_{[1,i]}$ and $X_{[2,i]}$ are the values of the two independent variables for observation i.
- β_1 and β_2 are the coefficients for X_1 and X_2, respectively, when the other variable has value 0.
- β_3 is the coefficient which controls the extent to which the coefficient on one variable depends on the value of the other.
- ϵ_i is the error term, assumed to be independent and normally distributed.

Bayesian formulation

In the Bayesian formulation, we define each parameter with priors . We can express the Bayesian regression model as follows:

$$Y_i \sim \text{Normal}(\alpha + \beta_1 X_{[1,i]} + \beta_2 X_{[2,i]} + \beta_3 X_{[1,i]} X_{[2,i]}, \sigma)$$

$$\alpha \sim \text{Normal}(0, 1)$$

$$\beta_1 \sim \text{Normal}(0, 1)$$

$$\beta_2 \sim \text{Normal}(0, 1)$$

$$\beta_3 \sim \text{Normal}(0, 1)$$

$$\sigma \sim \text{Exponential}(1)$$

Where:

- Y_i is the dependent variable for observation i.
- α is the intercept term, which in this case has a unit-normal prior.
- β_1 and β_2 are the coefficients for X_1 and X_2, respectively, when the other variable has value 0.

- β_3 is the coefficient which controls the extent to which the coefficient on one variable depends on the value of the other.
- $X_{[1,i]}$ and $X_{[2,i]}$ are the two values of the independent continuous variables for observation i.
- σ is a standard deviation parameter, which here has an Exponential prior that constrains it to be positive.

Model validation tests

- Comparison with Stan: `3.Continuous interactions.py` fits the same model with BayesForge and Stan and compares the posterior distributions (KL divergence).
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Notes

> **Note**
>
> The interaction term equation:
>
> $$Y_i \sim Normal(\alpha + \beta_1 X_{[1,i]} + \beta_2 X_{[2,i]} + \beta_3 X_{[1,i]} X_{[2,i]}, \sigma)$$
>
> can be re-written as:
>
> $$Y_i \sim Normal(\alpha + (\beta_1 + \beta_3 X_{[2,i]}) X_{[1,i]} + \beta_2 X_{[2,i]}, \sigma)$$
>
> simply by factoring the terms with $X_{[1,i]}$ in them. The result is that the coefficient on $X_{[1,i]}$ is written specifically as a linear regression model of $X_{[2,i]}$.

Reference(s)

McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian course with examples in R and Stan.* Chapman; Hall/CRC.